

Sistema de Gestión de Stock

Informe sobre el uso de Inteligencia Artificial en el desarrollo

Proyecto Django — Control de inventario, roles y movimientos de stock
Trabajo práctico — 2026

Este documento explica cómo usamos la Inteligencia Artificial mientras hacíamos el proyecto. Explicamos qué preguntas hicimos y cómo terminamos resolviendo cada cosa. La mayoría del proyecto lo armamos nosotros mismos, pensando la lógica y probando hasta que funcione. Solo en algunos puntos puntuales, que eran más difíciles, le pedimos ayuda a la IA para entender mejor algo o destrabarnos.

1. Qué hicimos nosotros solos

Nosotros pensamos directamente las siguientes partes del proyecto sin necesidad de consultar a la IA:

- Pensar las dos tablas principales: **Producto** y **Movimiento**, y cómo se relacionan entre sí.
- Decidir qué campos necesitaba cada tabla (nombre, cantidad, precio, tipo de movimiento, fecha, usuario).
- Armar las vistas (views.py) para el login, el dashboard, alta de productos, eliminar productos y registrar movimientos.
- Diseñar las pantallas HTML (login y dashboard) con Bootstrap, incluyendo los formularios y las tablas.
- Definir los tres roles del sistema (Admin, Operador, Consulta) y qué puede hacer cada uno.
- Conectar las URLs (urls.py) con cada vista.
- Probar el sistema manualmente: crear productos, hacer entradas y salidas, y revisar que el stock se actualice bien.

Estas partes las resolvimos charlando entre nosotros y mirando la documentación de Django. Recién cuando nos trabamos en algo puntual, ahí sí recurrimos a la IA, como se explica en el punto siguiente.

2. En qué partes puntuales usamos la IA

Hubo cuatro momentos concretos donde el proyecto se complicó más de lo esperado y ahí sí le pedimos una mano a la IA. Le preguntamos por qué pasaba algo o cómo pensar el problema, y después nosotros escribimos la solución.

2.1 — La restricción de horario para usuarios que no son Admin

Esta fue una de las partes mas difíciles del sistema: el Admin puede entrar a cualquier hora, pero el resto de los usuarios (Operador y Consulta) solo pueden ingresar entre las 08:00 y las 18:00. No sabíamos bien cómo comparar la hora actual del servidor con un rango de horarios en Python.

Qué preguntamos

Le pedimos a la IA que nos explique cómo comparar la hora actual con un rango de horas usando el módulo *datetime* de Python, y por qué conviene convertir todo a objetos de tipo *time* en vez de comparar strings de texto como "08:00" directamente.

Qué entendimos

Que comparar strings de texto a veces funciona por casualidad, pero no es seguro: si la hora cambia de formato, se rompe todo. Por eso usamos *datetime.strptime()* para convertir los horarios límite a un formato de hora real, y así poder compararlos con `<=` de forma confiable.

2.2 — Cómo organizar los permisos según el rol (Admin / Operador / Consulta)

Teníamos que lograr que cada parte del sistema (crear producto, eliminar producto, exportar CSV, registrar movimiento) se vea o no según el rol del usuario logueado. No sabíamos cuál era la forma correcta de chequear el rol del usuario tanto en las vistas (views.py) como en el HTML.

Qué preguntamos

Preguntamos cómo se usan los Grupos de Django para representar roles, y cómo consultar desde una vista o desde una plantilla HTML si el usuario logueado pertenece a un grupo determinado.

Qué entendimos

Que Django ya trae un sistema de Grupos (*Group*) que sirve justo para esto, y que se puede preguntar con `request.user.groups.filter(name='Admin').exists()`. Con esa idea armamos el control de roles en las vistas, y en el HTML usamos la variable `rol` que mandamos desde la vista para mostrar u ocultar botones y formularios.

2.3 — Generar el archivo CSV para descargar

Nunca habíamos generado un archivo descargable desde Django. No sabíamos cómo hacer que el navegador entienda que tiene que descargar un archivo en vez de mostrar texto plano en la pantalla.

Qué preguntamos

Le pedimos a la IA que nos explique para qué sirve el encabezado *Content-Disposition* en una respuesta HTTP, y cómo se usa el módulo `csv` de Python junto con un objeto *HttpResponse* de Django.

Qué entendimos

Que el *Content-Disposition: attachment* es lo que le dice al navegador "esto se descarga, no se muestra", y que el módulo `csv.writer()` puede escribir directamente sobre el objeto *HttpResponse* como si fuera un archivo común. Con esa base armamos la función `exportar_csv`.

3. Resumen de dificultad por parte del proyecto

Para que se vea de forma rápida, armamos esta tabla con las distintas partes del proyecto, cuánto nos costó cada una y si necesitamos ayuda de la IA o no.

Parte del proyecto	Dificultad	¿Usamos IA?	Comentario
Modelos (Producto, Movimiento)	Baja	No	Lo pensamos viendo la consigna y la doc oficial.
Login simulado y creación de usuarios	Media	No	Lo resolvimos probando con distintos usuarios.
Restricción de horario (8 a 18 hs)	Alta	Sí	Dudas con comparar horas en Python.
Roles con Grupos de Django	Media	Sí	No sabíamos cómo chequear el grupo del usuario.
Validar stock negativo	Media	Sí	Duda sobre el orden de validar vs. guardar.
Pantallas HTML con Bootstrap	Baja	No	Ya habíamos usado Bootstrap antes.
Exportar CSV	Alta	Sí	Primera vez generando un archivo descargable.
Panel de Administración (admin.py)	Baja	No	Solo era registrar los modelos.

4. Conclusión

Fuimos armando el proyecto desde las tablas, las vistas, las pantallas hasta las reglas de negocio. La IA nos sirvió como una consulta puntual donde no sabíamos un concepto técnico específico (horarios en Python, Grupos de Django, orden de validaciones y archivos descargables). En esos casos nos explicó el concepto y después nosotros escribimos la solución y la probamos. El resto del sistema lo pensamos y programamos sin ayuda externa.